Artificial Neural Networks (ANN)

1. What is an Artificial Neural Network (ANN)?

An Artificial Neural Network (ANN) is a computational model inspired by the human brain, designed to recognize patterns and learn from data. It consists of interconnected processing units called neurons that work together to solve tasks such as classification, regression, and prediction.

---

2. Biological vs Artificial Neuron

Biological Neuron	Artificial Neuron
Dendrites receive signals	Inputs (features)
Cell body processes signals	Weighted sum
Axon sends output	Activation function output

---

3. Structure of an ANN

An ANN is made up of layers:

1. Input Layer
   - Receives input features (X_1, X_2, ..., X_n)
   - No computation is performed
2. Hidden Layer(s)
   - Perform calculations using weights and activation functions
   - Extract complex patterns
3. Output Layer
   - Produces final prediction
   - Depends on task type:
     - Binary classification → Sigmoid
     - Multi-class classification → Softmax
     - Regression → Linear activation

Components of an Artificial Neuron

Each neuron performs the following operations:

1. **Weighted Sum**

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

2. **Activation Function**

$$a = f(z)$$

Where:

- w = weights
- x = inputs
- b = bias
- f = activation function

Where:

- (w) = weights
- (x) = inputs
- (b) = bias
- (f) = activation function

Ahmed

5. Weights and Bias

- Weights determine the importance of each input
- Bias allows the model to shift the activation function
- Both are learned during training

---

6. Forward Propagation

Forward propagation is the process where:

1. Input data flows from input layer to output layer
2. Weighted sums and activations are computed
3. Final output is generated

---

7. Loss (Cost) Function

The loss function measures how far the predicted output is from the actual output.
Common loss functions:

- Mean Squared Error (MSE) – Regression
- Binary Cross-Entropy – Binary classification
- Categorical Cross-Entropy – Multi-class classification

---

8. Learning Process in ANN

ANNs learn by:

1. Making predictions
2. Calculating loss
3. Updating weights using backpropagation
4. Repeating for multiple epochs

---

9. Advantages of ANN

- Can model complex non-linear relationships
- Works well with large datasets
- Automatically extracts features

---

10. Limitations of ANN

- Requires large amount of data
- Computationally expensive
- Less interpretable (black-box model)

---

11. Common Applications of ANN

- Image recognition
- Speech recognition
- Fraud detection
- Medical diagnosis
- Recommendation systems

Amar Ahmed

Activation Functions

1. What is an Activation Function?

An activation function decides whether a neuron should be activated or not by transforming the weighted sum of inputs into a non-linear output.

$$z = \sum w_i x_i + b, \quad a = f(z)$$

Without activation functions, a neural network behaves like a linear model, no matter how many layers it has.

---

2. Why Activation Functions are Important

- Introduce non-linearity
- Enable learning of complex patterns
- Control signal flow in deep networks
- Help gradients propagate during training

---

3. Common Activation Functions

(a) Sigmoid

$$f(z) = \frac{1}{1 + e^{-z}}$$

- Output range: (0, 1)
- Used in binary classification
- Vanishing gradient problem

(b) Tanh

$$f(z) = \tanh(z)$$

- Output range: (−1, 1)
- Zero-cantered
- Still suffers from vanishing gradients

(c) ReLU (Rectified Linear Unit)

Arthur Ahmed

$$f(z) = \max(0, z)$$

- Most widely used in hidden layers
- Fast computation
- Dying ReLU problem (neurons stuck at 0)

(d) Softmax

$$f(z_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

- Converts outputs into probabilities
- Used in multi-class classification

---

4. Activation Function Summary

Function	Output Range	Typical Use
Sigmoid	(0, 1)	Binary classification
Tanh	(−1, 1)	Hidden layers
ReLU	[0, ∞)	Hidden layers
Softmax	(0, 1), sum = 1	Multi-class output

Backpropagation

5. What is Backpropagation?

Backpropagation is the algorithm used to train neural networks by adjusting weights and biases to minimize the loss function.

It works by propagating the error backward from output layer to input layer.

---

6. Why Backpropagation is Needed

- Tells the network how much each weight contributed to the error
- Enables efficient learning in deep networks
- Uses gradient descent for optimization

---

7. Steps in Backpropagation

Step 1: Forward Pass

- Compute predictions using current weights
- Calculate loss using a loss function

Step 2: Compute Gradients

- Calculate partial derivatives of loss w.r.t. each weight
- Apply chain rule of calculus

Step 3: Update Weights

Athar Ahmed

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

Where:

- η = learning rate
- L = loss function

Step 4: Repeat

- Repeat for multiple epochs until loss is minimized

---

8. Intuition Behind Backpropagation

- If prediction is wrong, increase/decrease weights
- If prediction is close, make small adjustments
- Learning happens gradually

---

9. Problems in Backpropagation

- Vanishing gradients → slow learning (Sigmoid, Tanh)
- Exploding gradients → unstable training
- Solved using:
  - ReLU family
  - Proper weight initialization
  - Normalization techniques

---

10. One-Line short Answers

- Activation function:
  A function that introduces non-linearity into neural networks and determines neuron output.
- Backpropagation:
  An algorithm that updates neural network weights by propagating errors backward using gradient descent.

Amar Ahmed